

HTML - CSS

1. What is CSS?

- **CSS (Cascading Style Sheets)** is used to style and layout web pages.
- It allows you to control the appearance of HTML elements, such as colors, fonts, spacing, and positioning.

2. Ways to Add CSS to HTML

There are three ways to include CSS in an HTML document:

1. Inline CSS

- Applied directly to an HTML element using the `style` attribute.
- Example:

```
<p style="color: blue; font-size: 20px;">This is a styled paragraph.</p>
```

- **Tip:** Use inline styles sparingly, as they can make your HTML code harder to maintain.

2. Internal CSS

- Defined within the `<style>` tag in the `<head>` section of an HTML document.
- Applies to the entire page.
- Example:

```
<head>
  <style>
    p {
      color: red;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <p>This is a styled paragraph.</p>
</body>
```

- **Tip:** Use internal styles for single-page projects or small websites.

3. External CSS

- Stored in a separate `.css` file and linked to the HTML document using the `<link>` tag.
- Example:

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

In `styles.css`:

```
p {
  color: green;
```

```
font-size: 16px;
}
```

- **Tip:** Use external styles for larger projects to keep your HTML and CSS separate and organized.

Prioritate:

1. in line css
2. extern & intern
3. browser default

3. CSS Syntax

- CSS consists of **selectors** and **declaration blocks**.
- **Selector:** Targets the HTML element(s) to style.
- **Declaration Block:** Contains one or more declarations in the format `property: value; .`
- Example:

```
p {
  color: blue;
  font-size: 20px;
}
```

- `p` is the selector.
- `color: blue;` and `font-size: 20px;` are declarations.

4. Common CSS Properties

Here are some commonly used CSS properties:

Text Styling

- `color` : Sets text color.

```
p { color: red; }
```

- `font-size` : Sets text size.

```
p { font-size: 16px; }
```

- `font-family` : Sets the font.

```
p { font-family: Arial, sans-serif; }
```

- `text-align` : Aligns text (left, right, center, justify).

```
p { text-align: center; }
```

Background Styling

- `background-color` : Sets background color.

```
body { background-color: lightblue; }
```

- `background-image` : Sets a background image.

```
body { background-image: url("image.jpg"); }
```

Box Model

- `margin` : Adds space outside an element.

```
p { margin: 10px; }
```

- `padding` : Adds space inside an element.

```
p { padding: 15px; }
```

- `border` : Adds a border around an element.

```
p { border: 2px solid black; }
```

Layout

- `display` : Controls how an element is displayed (e.g., `block` , `inline` , `flex`).

```
div { display: flex; }
```

- `position` : Controls positioning (e.g., `static` , `relative` , `absolute` , `fixed`).

```
div { position: relative; }
```

5. CSS Selectors

- **Element Selector**: Targets HTML elements by tag name.

```
p { color: blue; }
```

- **Class Selector**: Targets elements with a specific class (prefixed with `.`).

```
.highlight { background-color: yellow; }
```

- **ID Selector**: Targets a single element with a specific ID (prefixed with `#`).

```
#header { font-size: 24px; }
```

- **Grouping Selectors**: Applies styles to multiple selectors.

```
h1, h2, h3 { color: green; }
```

6. CSS Comments

- Comments in CSS are written using `/* ... */`.
- Example:

```
/* This is a CSS comment */  
p { color: blue; }
```

7. Tips from W3Schools

1. **Use External CSS**:

- External stylesheets are the best practice for larger projects. They keep your HTML clean and make it easier to maintain and update styles across multiple pages.

2. **Avoid Inline Styles**:

- Inline styles mix content with presentation, making your code harder to read and maintain. Use them only for quick fixes or specific cases.

3. Use Shorthand Properties:

- CSS shorthand properties can make your code cleaner and more efficient. For example:

```
/* Longhand */
margin-top: 10px;
margin-right: 20px;
margin-bottom: 10px;
margin-left: 20px;

/* Shorthand */
margin: 10px 20px;
```

4. Organize Your CSS:

- Group related styles together and use comments to separate sections. For example:

```
/* Header Styles */
header { background-color: #333; color: white; }

/* Navigation Styles */
nav { display: flex; justify-content: space-between; }
```

5. Use CSS Variables:

- CSS variables (custom properties) make it easier to reuse values throughout your stylesheet.

```
:root {
  --primary-color: blue;
}
p {
  color: var(--primary-color);
}
```

6. Test Cross-Browser Compatibility:

- Different browsers may render CSS differently. Always test your website on multiple browsers to ensure consistency.

7. Use Responsive Design:

- Use media queries to create responsive designs that work on all devices.

```
@media (max-width: 600px) {
  body {
    font-size: 14px;
  }
}
```

- display:
 - inline – Elementul rămâne pe aceeași linie cu alte elemente, respectând dimensiunea conținutului său. Exemplu: `<span>`, `<a>`, `<cite>`, `<em>`, `<input>`, `<label>`, `<ing>`
 - block – Elementul ocupă întreaga lățime disponibilă și începe pe un rând nou. Exemplu: `<div>`, `<p>`, `<h1>`.
 - none – Elementul nu este afișat deloc (dispare complet din fluxul paginii).
 - none vs visibility "hidden" vs input type "hidden"

- `flex` – Transformă elementul într-un container flexbox, permițând aranjarea flexibilă a copiilor săi (`display: flex;` activează Flexbox).

Diferențele dintre `display: none`, `visibility: hidden` și `<input type="hidden">`:

1. `display: none`

- Elementul nu este afișat și nu ocupă spațiu în pagină.
- Este eliminat complet din fluxul documentului.
- Ex.:

```
.hidden {  
    display: none;  
}
```

```
<div class="hidden">Acest text nu se vede și nu ocupă spațiu.</div>
```

2. `visibility: hidden`

- Elementul este invizibil, dar încă ocupă spațiu în pagină.
- Diferența față de `display: none` este că structura paginii rămâne neschimbată.
- Ex.:

```
.invisible {  
    visibility: hidden;  
}
```

```
<div class="invisible">Acest text nu se vede, dar spațiul său rămâne rezervat.</div>
```

3. `<input type="hidden">`

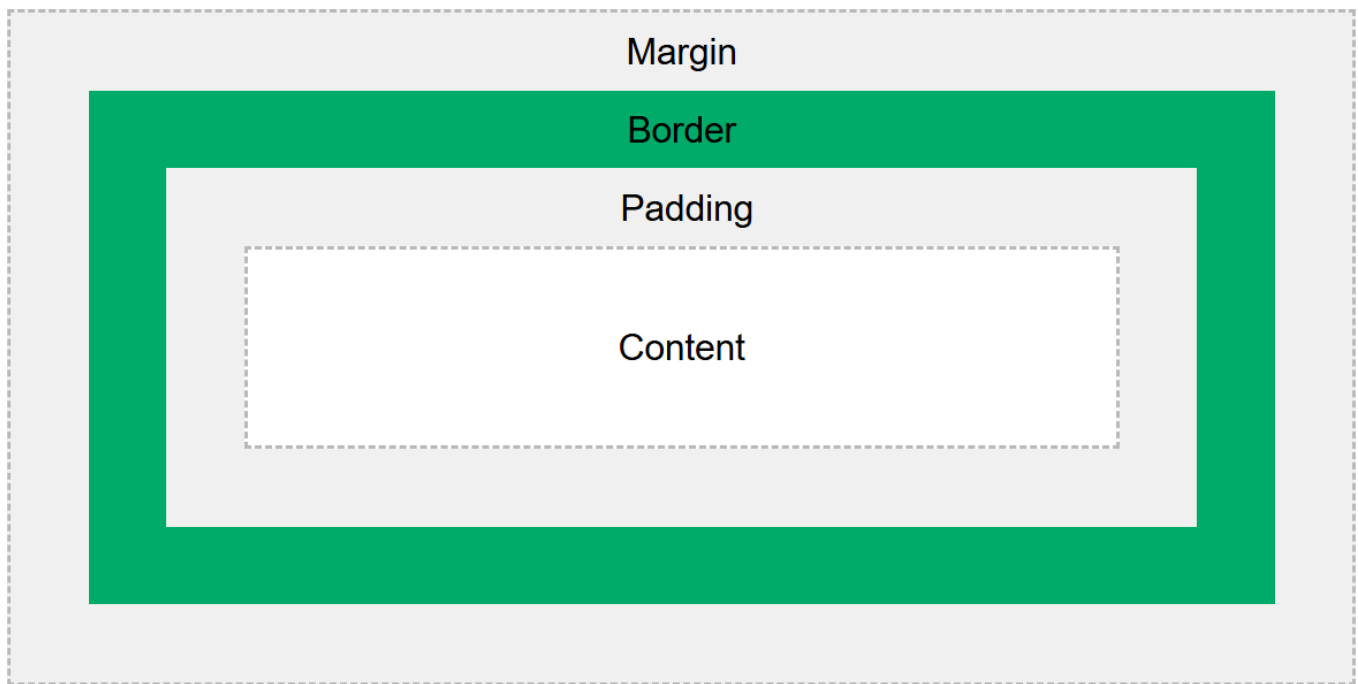
- Este un element de tip input care nu este vizibil pentru utilizator, dar există în pagină și poate fi folosit în formulare.
- Nu poate fi afișat cu CSS.
- Folosit pentru a transmite date în formulare fără ca utilizatorul să le vadă.
- Ex.:

```
<form action="submit.php" method="post">  
    <input type="hidden" name="user_id" value="12345">  
    <button type="submit">Trimite</button>  
</form>
```

◆ Când să folosești fiecare?

- ✓ `display: none` – Când vrei să ascunzi complet un element și să nu ocupe spațiu.
- ✓ `visibility: hidden` – Când vrei ca un element să fie invizibil, dar să păstreze spațiul său.
- ✓ `<input type="hidden">` – Când trebuie să trimiți date printr-un formular fără ca utilizatorul să le vadă.

The box model CSS



- element width
- padding
- border
- outline
- margin

Selectori

- tag-ul: `p`
- `.clasa` : `table` (se pot adauga mai multe clase)
- `#id`
- - `:` : selectorul universal

Cominator select